SIMATS ENGINEERING

CSA08 - Python Programming

ASSIGNMENT 5

CO5

Data Acquisition, Cleaning, Analysis and AI Applications

Total Marks: 50 | Individual Assignment

Solutions for A1-A5 and B1-B4

Student Name: PROBIR SAHA SHOHOM

Register Number: 1911260319

GitHub Repository Link:
https://github.com/Probir127/CSA08-1911260319-Assignment5

Submission details and scope

Student: PROBIR SAHA SHOHOM. Register number: 1911260319. Course: CSA08 - Python Programming. Total marks: 50. Repository: https://github.com/Probir127/CSA08-1911260319-Assignment5.

This report covers A1-A5 (25 marks) and B1-B4 (25 marks), including all three Complete the Code exercises. It follows acquire, inspect, clean, normalise, store, query, visualise, profile and interpret. The code, source fixtures, generated charts, SQLite database, raw timings and output transcripts accompany the report.

All datasets are explicitly synthetic. Measurements were actually executed in the recorded local environment; they vary across hardware and runs. The student should review, personally run and understand the work, capture the execution screenshots required by the brief, complete their own complaint-label review and share the repository with the faculty. The included output images render captured stdout and are labelled accordingly; they are not terminal screenshots.

Part A uses structured ALGORITHM, INPUT, OUTPUT, BEGIN/END, IF/ENDIF and FOR/ENDFOR pseudocode. The source names a SIMATS standard but does not attach its detailed template, so exact conformity to an unavailable institutional template is not claimed. The algorithms are language-independent; Python source is provided separately for Part B.

Environment and evidence map

Python 3.12.14; pandas 2.2.3; NumPy 2.3.5; matplotlib 3.11.2; SQLite 3.53.1; macOS 27.0 on arm64. requirements.txt pins the external libraries actually used. Standard-library modules include csv, io, sqlite3, statistics, re, logging, time, timeit, cProfile, pstats, resource, subprocess, pathlib, datetime and json.

Task	Main deliverables
A1-A5	One task folder per design, pseudocode and complexity comparison
B1	pipeline.py, demo.py, complete_code.py, messy/clean CSVs, SQLite, four charts, timings
B2	adversarial.py, complete_code.py, malformed fixtures, SQL and logging experiments
B3	benchmarks.py, generator, three-size timings, full profile, agreement checks
B4	ai.py, complete_code.py, 50 complaints, paired summaries, chatbot, index experiment

Task A1 - Design the data acquisition (7 marks)

A1.1 Source structure and expected problems

One UTF-8 CSV contains a header followed by hourly readings. The exact columns are station,date,hour,pm25,pm10,no,temperature,temp_unit. One example is Adyar,2026-10-01,0,48,80,15,30,C. PM2.5, PM10 and NO use micrograms/m3; NO denotes nitric oxide, not NO2. Temperature uses the declared C or F unit. The sample network has 12 stations, with Guindy and Ambattur configured for F. All measurements and complaints supplied with this solution are synthetic teaching fixtures, not observations from the city.

The added station and unit columns make the source self-describing. In a real deployment, a trusted station manifest could supply them when the incoming CSV omits them. That adapter must validate the filename/station mapping rather than infer a unit from how large a temperature looks.

Problem	Example	Implemented treatment / evidence
Missing number	blank PM2.5; PM10 = NA	Reject incomplete row; small_messy.csv
Negative pollutant	NO = -1	Reject; concentrations cannot be negative

Problem	Example	Implemented treatment / evidence
Implausibly high value	PM2.5 = 1000; temperature = 61 C	Reject against declared sensor limits
Non-numeric or non-finite	broken, nan, inf	Reject before arithmetic
Duplicate key	Adyar, 2026-10-01, hour 0 repeated	Keep first valid row and log duplicate
Date malformed or impossible	01/10/2026; 2026-02-30	Require a valid ISO calendar date
Hour invalid	24; 1.5	Require integer 0 through 23
Station whitespace/case	two spaces, aDyAr, two spaces	Canonicalise to Adyar through registry
Unknown station	Unknown or empty	Reject, rather than invent a station
Fahrenheit or unit conflict	Guindy 86 F; Adyar declared F	Convert registered F to 30 C; reject conflict
Row-width mismatch	missing unit; extra ninth field	Reject and log row
Header/delimiter wrong	missing pm10; semicolons	Reject whole file, explicit SourceError
Wrong encoding	Latin-1 accented station	Reject file as UTF-8 decoding failure
Empty file / invalid CSV syntax	no header; unclosed quote	Reject whole file
Quote in legitimate station	King's Park	Accept; bind value in SQL parameters

Malformed records, missing columns, wrong delimiters and encoding problems are separately exercised in B1 and B2. Empty lines are skipped by csv.DictReader; they are not observations. Thousands separators and localized date formats are outside this source contract and are not silently guessed.

A1.2 SQLite design and mapping

The stations table has station TEXT PRIMARY KEY and temperature_unit TEXT NOT NULL constrained to C/F. The readings table has station TEXT NOT NULL as a foreign key, day TEXT NOT NULL, hour INTEGER NOT NULL, pm25 REAL NOT NULL, pm10 REAL NOT NULL, no REAL NOT NULL, and temperature_c REAL NOT NULL. Its composite primary key is (station, day, hour); SQL CHECK constraints reinforce hour and measurement ranges. Foreign keys are enabled for the load connection.

CSV station maps through the canonical registry; date maps to day; hour maps to an integer; three pollutants map to finite REAL values; temperature/temp_unit map to temperature_c after conversion. Unit metadata belongs to stations and is not repeated in each cleaned reading. Raw files and a separate JSON rejection ledger preserve the original input and reasons. Dates are validated by Python before insertion; TEXT alone does not enforce a calendar date.

A daily_summary view groups readings by station and day. A production ingestion ledger should additionally store source file, checksum, ingestion time and rule version. The demonstration loader deliberately rebuilds its own output database on each run; it is not an incremental production loader.

```
ALGORITHM ACQUIRE_AND_STORE
INPUT: source file, station registry, destination database
OUTPUT: validated readings, rejection ledger, database
BEGIN
    OPEN source AS UTF-8 CSV
    IF header does not match required fields THEN
        REPORT file error
        STOP
    ENDIF
    INITIALISE accepted records, rejection ledger, seen keys
    FOR EACH numbered source record DO
        VALIDATE AND NORMALISE record using cleaning rules
        IF record is invalid THEN
            APPEND line and reason TO rejection ledger
        ELSE IF record key is in seen keys THEN
            APPEND duplicate reason TO rejection ledger
```

```
        ELSE
            ADD record key TO seen keys
            APPEND cleaned record TO accepted records
        ENDIF
    ENDFOR
    CREATE stations and readings tables
    INSERT station metadata using bound parameters
    INSERT accepted records using one batch operation
    COMMIT transaction
    RETURN accepted records, rejection ledger, database
END
```

A1.3 Three board questions: CSV versus SQL

Question 1: What is Adyar's average PM2.5 in October 2026? CSV processing scans every relevant file, filters station/date, and maintains sum/count. SQL uses AVG(pm25), COUNT(*) with station=? AND day>=? AND day<?. The half-open interval ends at 2026-11-01, avoiding accidental November data.

Question 2: How do stations compare in October? CSV uses a dictionary of station sums/counts, or pandas groupby. SQL filters the month, groups by station, and orders AVG(pm25) descending. Question 3: Which five hours have the largest October mean PM2.5? CSV groups by hour; SQL GROUP BY hour orders the mean descending and takes LIMIT 5. A tie uses hour ascending in the B1 query.

Loading all CSVs into a pandas DataFrame for every question costs O(n) parsing time and O(n) working memory, before the aggregation; materialized copies increase constants. A streaming CSV sum/count answer can use O(1), O(stations), or O(24) auxiliary memory, but still scans O(n) records. SQLite holds O(n) rows plus indexes on disk and can return small result sets without loading the entire table into Python.

The assignment's approximate O(log n) index lookup is only the search component. A station/month average must also read k matching values: roughly O(log n + k) with an appropriate index/covering access path, not O(log n) for the entire average. A station-leading index does not automatically accelerate a date-only city-wide query. B4 tests that distinction; plan details and all query results are saved. SQLite's primary-key index already begins with (station, day), so a duplicate index on that prefix is unnecessary in B1.

Task A2 - Design the cleaning and normalisation (5 marks)

A2.1 Rules and justification

The chosen baseline drops a row if any required numeric reading is blank, NA, malformed, non-finite or outside the sensor range. Keeping only complete observations makes downstream comparisons consistent and avoids treating invented values as measured concentrations. It can bias averages if failures occur during pollution spikes; therefore every daily mean must be accompanied by the valid count and coverage. The alternative of retaining partially valid rows with per-column counts is useful, but is not the implemented complete-case policy.

Assumed sensor acceptance limits are PM2.5 0..999, PM10 0..1999, NO 0..999 micrograms/m3, and temperature -50..60 C after conversion. These are demonstration validation bounds, not health thresholds or verified specifications for real instruments. Missing or impossible values are not replaced with zero. An unknown station, invalid timestamp or unit conflict rejects the record because it cannot be attributed reliably.

A duplicate means the same canonical station, date and hour, even if the readings differ. Keep the first valid row in source order and log later duplicates; an earlier invalid row never reserves its key. A real correction protocol should retain versions instead of blindly overwriting data. Fahrenheit conversion is C = (F - 32) * 5 / 9; both the row unit and trusted station metadata must agree.

Forward fill could retain a station's last valid value, only after sorting its readings by timestamp and limiting the gap length; never fill across stations. It risks hiding abrupt changes. A station/month median is less sensitive to outliers than a mean but flattens variation and can leak future information into an online task. A training model must fit imputation statistics only on its training window. Any imputed values should carry flags.

A2.2 Min-max and z-score

For finite values x, min-max scaling is (x - minimum)/(maximum - minimum); z-score is (x - mean)/population_standard_deviation. Min-max is useful when a model expects comparable bounded training features; z-score expresses distance from the mean and is useful when relative variation matters. Neither method makes a skewed distribution normal or repairs bad data, and min-max is sensitive to outliers.

The functions return an empty list for no data and zeros for a constant series, avoiding division by zero. Fit a shared reference interval when comparing stations; independently rescaling each station would erase absolute differences. Save the fitted parameters for future inputs; future min-max values can fall outside 0..1 and should be investigated, not silently clipped. Raw concentrations remain in SQLite; scaling is a derived analysis feature, not a replacement for units.

```
ALGORITHM NORMALISE_PM25
INPUT: validated finite values
OUTPUT: scaled values and standardised values
BEGIN
    IF there are no values THEN
        RETURN two empty lists
    ENDIF
    COMPUTE minimum, maximum, mean and population deviation
    FOR EACH value DO
        IF maximum equals minimum THEN SET scaled TO 0
        ELSE SET scaled TO (value - minimum) / (maximum - minimum)
        ENDIF
        IF deviation equals 0 THEN SET standardised TO 0
        ELSE SET standardised TO (value - mean) / deviation
        ENDIF
        APPEND results TO output lists
    ENDFOR
    RETURN output lists
END
```

A2.3 Standard formatting and alternative complexity

Use YYYY-MM-DD dates, local integer hours 0..23, station registry spelling, UTF-8 CSV with a header, Celsius and the stated pollutant units. Retain full floating-point precision for calculation; display means and charts to sensible precision and CSV summaries to four decimal places. Binary floats are approximate, so correctness comparisons use a tolerance.

Dropping costs O(n) after parsing; streaming rejection can use O(1) auxiliary state, although this implementation retains accepted rows, seen keys and errors and therefore uses O(n) memory. Forward fill after timestamp ordering is O(n), with O(stations) last-value state; sorting unsorted inputs adds O(n log n) time and typically O(n) storage. A median obtained by sorting costs O(m log m) for each station group and O(n) total retained values; selection algorithms can do better, but that is not the sorting-based alternative requested.

For observations [10, missing, missing, 90], dropping gives mean 50 with only 2/4 valid samples; forward fill gives [10,10,10,90] and mean 30; median fill gives [10,50,50,90] and mean 50. None proves the missing observations were 10 or 50. This example shows why the report records coverage and does not call imputed values valid measurements.

Task A3 - Design the analysis and visualisation (5 marks)

A3.1 Chart choices

Board question	Chart and axes	Reason
PM2.5 trend over a month	Line: date versus daily mean PM2.5; separate Adyar/Guindy lines	Preserves chronological order and makes rises/falls visible without 12 overlapping traces
Compare stations	Horizontal bar: station versus monthly mean PM2.5	Direct ranking; long station names remain readable
Temperature versus PM2.5	Scatter: Celsius versus PM2.5 for Adyar	Shows spread and possible association; restricting station reduces one source of confounding
Hours with worst air	Box plot: local hour versus PM2.5 distribution	Shows median, variability and overlap for each hour; complements the top-five mean query

A histogram of concentrations would answer how often concentrations occur, not which clock hours have the worst air. An hourly bar of means is another reasonable choice, but it hides spread; the box plot is used here. Label data as synthetic and retain physical units. Do not infer causation, health risk categories, or real-city trends from these generated charts.

A3.2 Daily summary table

Each row is keyed by station/day and contains minimum, maximum, mean and valid_count. The implemented daily_summary view derives these fields through MIN, MAX, AVG and COUNT over accepted records. The CSV is ordered by station/day. Hourly full-day expected_count is 24, so coverage is valid_count/24 when the station was scheduled for the entire day. A future production summary should join a station/day calendar to display zero-count days explicitly; the current view produces no row for a day with zero accepted readings.

For missing readings, count means accepted observations, not scheduled observations. A network-wide daily mean weights the available readings; an equal-station average is a different statistic and must be named separately. If daily summaries are combined into a monthly mean, weight by their counts or retain sum/count; taking an unweighted average of daily means can give the wrong answer when coverage differs.

```
ALGORITHM DAILY_SUMMARY
INPUT: validated records
OUTPUT: summary keyed by station and day
BEGIN
    INITIALISE empty group dictionary
    FOR EACH record DO
        SET key TO record station and day
        IF key is new THEN
            INITIALISE sum TO 0, count TO 0
            INITIALISE minimum AND maximum TO record PM2.5
        ENDIF
        ADD record PM2.5 TO group sum
        INCREMENT group count
        UPDATE group minimum AND maximum
    ENDFOR
    FOR EACH group DO
        SET mean TO sum / count
        OUTPUT station, day, minimum, maximum, mean, count
    ENDFOR
END
```

The running-dictionary version is expected O(n) time and O(g) group state for g station/day groups. Reading all rows and storing lists before statistics.mean remains O(n) time but O(n) values plus O(g) metadata. SQL GROUP BY must also inspect relevant values; it is not automatically O(log n). Depending on its plan, ordered grouping can scan O(n), while sorting may cost O(n log n) with temporary storage that can spill to disk. B3 measures the actual SQLite plan's workload rather than assuming a universal winner.

Task A4 - Plan debugging and profiling (4 marks)

A4.1 Debugging plan

Start with the hand-checkable four accepted records and seventeen rejects. Print a small raw/clean pair and reject reason while diagnosing a defect. Use assertions for developer invariants: known 86 F -> 30 C, normalisation of a constant series, query agreement, valid counts, and expected fixture results. Assertions supplement explicit input validation; they can be disabled with optimisation, so input safety must never depend on them.

The pipeline uses a named logger for accepted/rejected counts, while structured ledgers preserve row numbers and reasons. Configure INFO during a diagnostic run and WARNING for routine operation. Log lazily with placeholders to avoid eager formatting when the message is disabled; avoid logging an entire sensitive complaint in a production system. Set a debugger breakpoint inside the rejected-row handler, inspect raw/line/unit/temperature, and step through conversion or date validation. The supplied runnable checks are the evidence; an interactive pdb session is a plan, not a claimed captured execution.

```
ALGORITHM DEBUG_A_REJECTED_READING
INPUT: smallest failing source record and expected outcome
```

```
OUTPUT: explained defect or confirmed rejection
BEGIN
    REPRODUCE failure using the small input
    PRINT only the raw record and observed outcome
    SET breakpoint immediately before validation
    INSPECT parsed fields, station metadata and conversion result
    STEP through the first failing condition
    COMPARE observed behaviour WITH the explicit cleaning rule
    CORRECT the implementation only if the rule is violated
    RUN the small regression check and the complete pipeline checks
    KEEP rejection counts in logging; remove temporary prints
END
```

A4.2 Profiling plan and print-only alternative

Use time.perf_counter around complete operations with clear boundaries. Use timeit.repeat or repeated timeit.timeit for small stable functions, keeping data generation and imports outside the timed block. Use cProfile on the plain-Python daily mean calculation, then inspect both cumulative time and internal time; a top-level wrapper's cumulative time includes its children and does not identify the deepest bottleneck by itself.

B1/B3 run three fresh subprocess trials per size/method and report medians plus all raw values; import time is excluded, but process peak RSS includes imports. B2 runs five insertion trials, three timeit trials of 10,000 debug calls, and compares identical transaction policies. B4 warms each query and uses seven trials before/after indexing. These are local repeated measurements, not confidence intervals or guarantees for another computer.

A print-only solution can use IF debug_enabled THEN PRINT state, but offers less selective control than logger levels and cannot inspect the live stack like a debugger. n fixed-length printed messages add O(n) work and O(total message bytes) output; terminal rendering can dominate runtime. Enabled logging also formats and emits O(n) messages and may cost more per call. Disabled logging still incurs n function/level checks, and arguments evaluated before the call still cost time. For very expensive diagnostic preparation, guard it with isEnabledFor or a debug flag. Remove temporary print calls or guard them; switch logging off by raising the level, and remove breakpoints before routine execution. The B2 timing uses StringIO, so it does not claim to measure interactive terminal latency.

Task A5 - Design an AI application for citizen complaints (4 marks)

A5.1 Sentiment and extractive summary

The sentiment baseline tokenises lowercase words and assigns +1 to good, clean, fast, helpful and improved; -1 to bad, dirty, slow, rude, smoke and broken. A directly preceding not, never or no flips the word's sign. Positive totals give positive, negative totals negative, and zero neutral. The practical function keeps sentence boundaries, so a negator cannot cross a sentence; the separately completed exercise preserves the supplied simpler tokenisation.

For an extractive summary, split the complaint into sentences, remove a declared set of stop words from each, count important-word frequencies across that complaint, and score a sentence by the average frequency of its important words. Select the top k with original position as the tie-breaker, then restore original order. Frequency weighting is a heuristic: repeated boilerplate can outrank a critical sentence. Extraction copies source sentences and does not invent new wording, but can omit context, negation or the speaker's main concern.

```
ALGORITHM SENTIMENT_SCORE
INPUT: complaint, positive words, negative words, negators
OUTPUT: sentiment label and numeric score
BEGIN
    SET total TO 0
    FOR EACH sentence IN complaint DO
        TOKENISE sentence into lowercase words
        FOR EACH word with its position DO
            SET value TO 1 if positive, -1 if negative, otherwise 0
            IF preceding word is a negator THEN
                SET value TO negative value
            ENDIF
            ADD value TO total
        ENDFOR
    ENDFOR
    RETURN positive if total > 0, negative if total < 0, otherwise neutral
```

```
END

ALGORITHM EXTRACTIVE_SUMMARY
INPUT: complaint and requested sentence count k
OUTPUT: selected original sentences
BEGIN
    SPLIT complaint into sentences
    IF k <= 0 THEN RETURN empty summary
    ENDIF
    TOKENISE each sentence and REMOVE stop words
    COUNT important-word frequencies across the complaint
    COMPUTE each sentence's average important-word frequency
    RANK sentences by decreasing score, then original position
    SELECT at most k sentences and RESTORE source order
    RETURN selected source sentences
END
```

A5.2 Rule-based chatbot and limitations

The chatbot accepts PM2.5/PM25, exactly one registered station and either today or an ISO date. It rejects ambiguous station/date requests, validates the calendar date, and executes a parameterised average/count query. An empty result becomes 'No valid readings', never zero pollution. Its reply includes the station, date, mean, physical unit and count. The demonstration fixes today to 2026-10-01 solely for reproducibility; normal execution uses the current date.

```
ALGORITHM ANSWER_PM25_QUESTION
INPUT: question, station registry, database, current date
OUTPUT: factual answer or clarification
BEGIN
    IF PM2.5 intent is absent THEN RETURN supported-question guidance
    ENDIF
    MATCH registered station names at word boundaries
    IF not exactly one station matches THEN RETURN clarification
    ENDIF
    RESOLVE one explicit valid date OR today
    IF a date cannot be resolved THEN RETURN clarification
    ENDIF
    QUERY average and count using bound station and date parameters
    IF count equals 0 THEN RETURN no-valid-readings message
    ENDIF
    RETURN station, date, mean, unit and valid count
END
```

This system does not understand arbitrary synonyms, Tamil or other languages, misspellings, yesterday/last week, subtle negation, sarcasm, or causal and health questions. The demonstration returns a daily mean, not the latest instantaneous measurement. A complaint with both positive and negative aspects can sum to neutral even when its final opinion is clearly negative.

A trained Naive Bayes alternative would collect independently labelled examples, split training/validation/test data, fit a tokenizer and vocabulary, estimate class priors and token likelihoods, then evaluate macro-F1 and per-class errors on held-out complaints. Inference is roughly O(W*C) for W tokens and C classes, with O(V*C) model memory. The fixed lexicon is O(W) time and O(V_lexicon) dictionary memory, aside from token lists; it needs no training. A language model uses substantially more model memory and compute, with costs depending on architecture and input length. Better accuracy is possible, not guaranteed; domain, language, annotation quality and distribution shift must be tested. No trained-model accuracy is claimed here because no labelled city corpus or model experiment was supplied.

Task B1 - Implement and validate the cleaning pipeline (7 marks)

B1.1 Before/after evidence

TaskB1/small_messy.csv has 21 records. Its first record is Adyar,2026-10-01,0,48,80,15,30,C; the second uses aDyAr with surrounding spaces and a lowercase unit; the third is Guindy at 86 F; the fourth is King's Park. The remaining 17 records deliberately introduce each row-level fault listed in A1.1. The original rows, accepted rows and full rejection messages are preserved in small_before_after.json, and small_cleaned.csv is directly inspectable.

Accepted records after canonicalisation and conversion:

Station	Date/hour	PM2.5	PM10	NO	Celsius
Adyar	2026-10-01/0	48.00	80.00	15.00	30.00
Adyar	2026-10-01/1	60.00	90.00	20.00	31.00
Guindy	2026-10-01/0	95.00	150.00	25.00	30.00
King's Park	2026-10-01/0	40.00	70.00	10.00	29.00

Rejected records (line numbers include the header):

CSV line	Reason
6	missing pm25
7	missing pm10
8	no outside 0..999
9	pm25 outside 0..999
10	could not convert string to float: 'broken'
11	non-finite pm25
12	non-finite pm25
13	temperature outside -50..60 C
14	day is out of range for month
15	Invalid isoformat string: '01/10/2026'
16	hour outside 0..23
17	hour must be an integer
18	unknown or empty station
19	temperature unit disagrees with station metadata
20	duplicate station/day/hour; first valid row retained
21	wrong number of columns
22	wrong number of columns

All 17 faulty records are rejected; all four expected valid records are retained. Checks also cover empty/constant scaling, exact Fahrenheit conversion, a case-normalised station, and z-score mean approximately zero. For accepted PM2.5 values [48,60,95,40], min-max gives approximately [0.145455,0.363636,1.000000,0.000000]. The normalisation functions preserve full precision; display rounding is separate.

B1.2 Implementation compared with the design

The design's complete-case rejection, source-order duplicate policy, trusted station units and separate rejection ledger are implemented without changing their meaning. The file reader uses UTF-8 with optional BOM support; a broken header, CSV quoting or encoding fails the entire acquisition call instead of returning a silently partial file. The small script retains rows and errors in memory to make the evidence reviewable, so its memory is O(n); a production implementation should write batches and stream its error ledger.

The specified Complete the Code exercise is deliberately separate: it has only station/date/pm25 columns and filters valid non-negative PM2.5. Its TODO regions are filled, the original example output matches, and the required above-999 assertion uses an added 1000 test row. It does not pretend to be the full eight-column production validator.

```
[('Adyar', '2026-10-01', 48.0), ('Guindy', '2026-10-01', 95.0), ('Guindy', '2026-10-03', 121.0)]
[0.0, 0.644, 1.0]
Above-999 row skipped: PASS
```

B1.3 Store and query

The monthly generator creates 8,928 hourly observations for 12 stations over 31 days. Forty-eight contain missing PM2.5; 8,880 are accepted. The database contains 372 station/day summaries with 23 or 24 accepted readings per day. The stored units and checks follow A1, and executemany inserts all accepted records in a transaction.

The three implemented SQL questions are:

```
SELECT AVG(pm25), COUNT(*) FROM readings
WHERE station = ? AND day >= ? AND day < ?;

SELECT station, AVG(pm25), COUNT(*) FROM readings
WHERE day >= ? AND day < ?
GROUP BY station ORDER BY AVG(pm25) DESC;

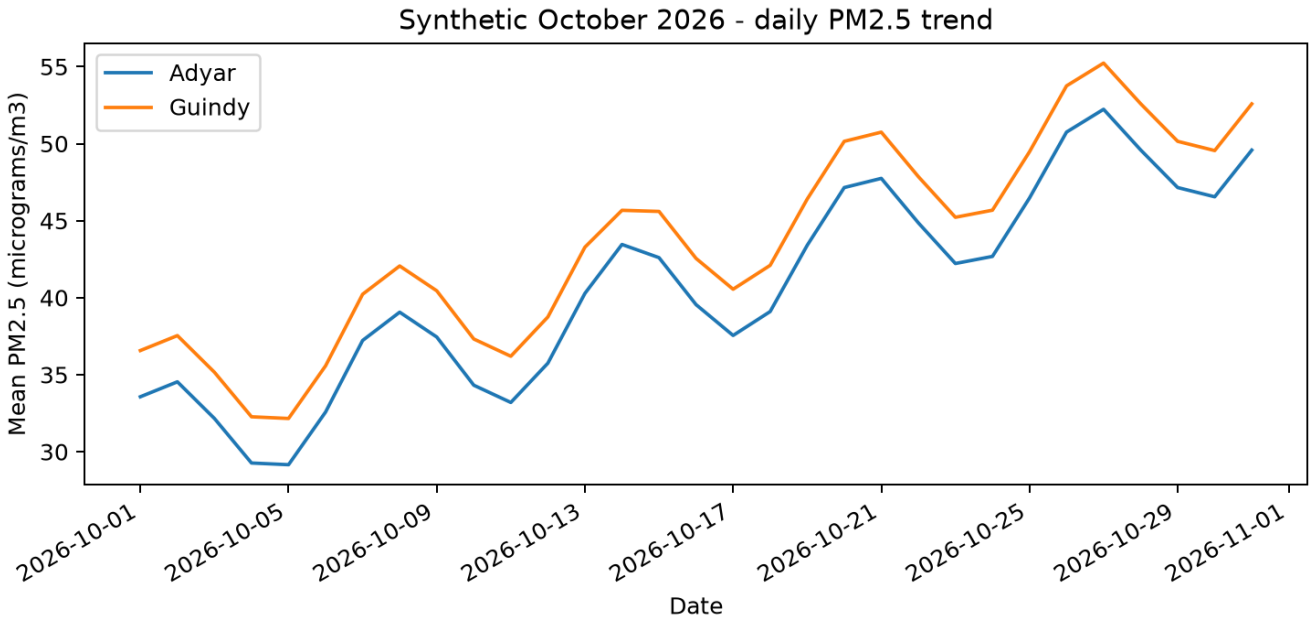
SELECT hour, AVG(pm25), COUNT(*) FROM readings
WHERE day >= ? AND day < ?
GROUP BY hour ORDER BY AVG(pm25) DESC, hour LIMIT 5;
```

Bind Adyar/2026-10-01/2026-11-01 for question 1 and the same two date bounds for questions 2 and 3. Adyar's monthly mean is 40.6632 micrograms/m3 over 740 readings. The station comparison is:

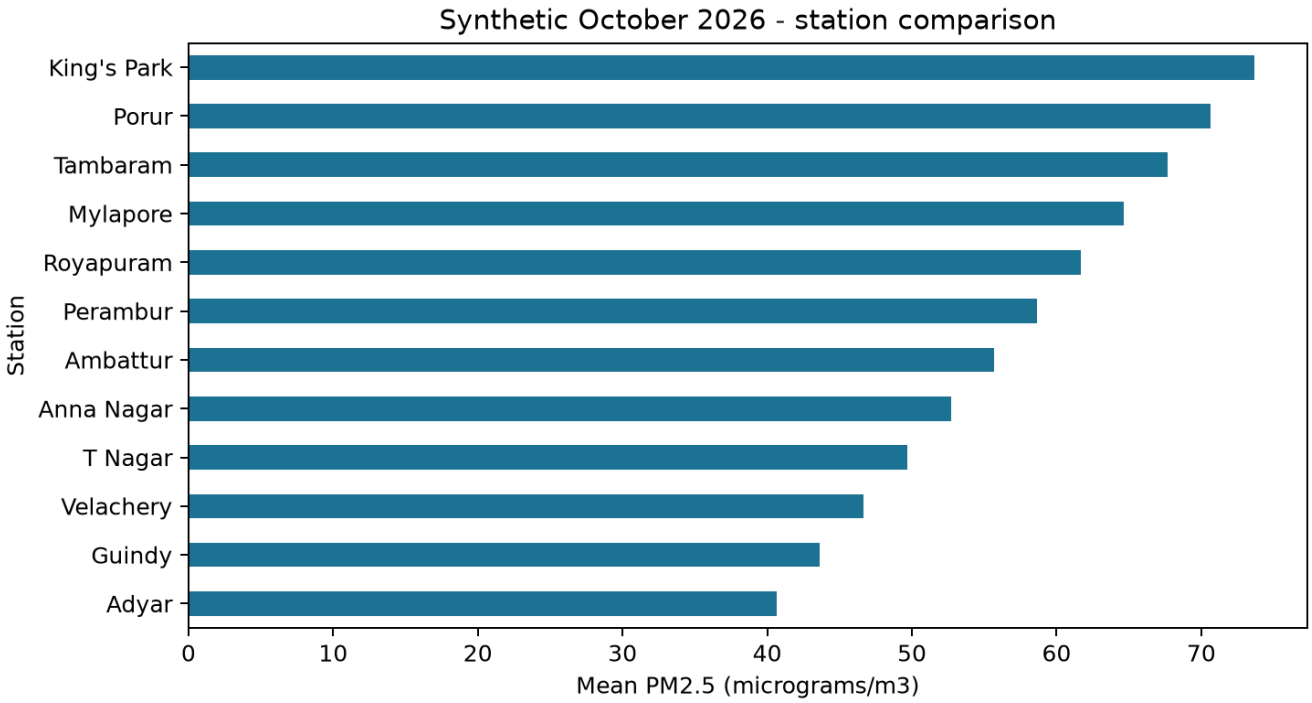
Station	October mean PM2.5	Valid readings
King's Park	73.6489	740
Porur	70.6692	740
Tambaram	67.6692	740
Mylapore	64.6692	740
Royapuram	61.6692	740
Perambur	58.6692	740
Ambattur	55.6875	740
Anna Nagar	52.6875	740
T Nagar	49.6632	740
Velachery	46.6389	740
Guindy	43.6389	740
Adyar	40.6632	740

The top five mean hours are 20, 19, 8, 18 and 7, respectively, with means 70.6963, 70.6801, 70.6746, 70.6639 and 70.6584 micrograms/m3. Every one has 370 valid observations across stations/days. The tiny ranking differences reflect the deterministic missing-value pattern; hour 9 also belongs to the generator's high-reading band.

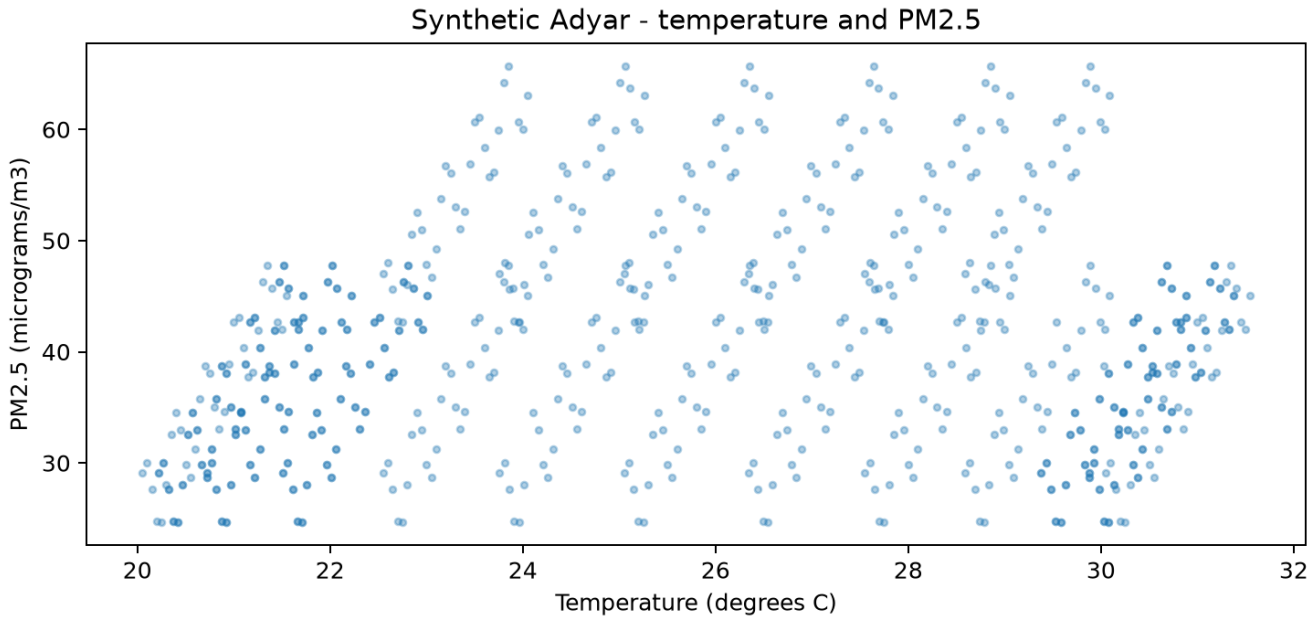
B1.4 Four charts and interpretation



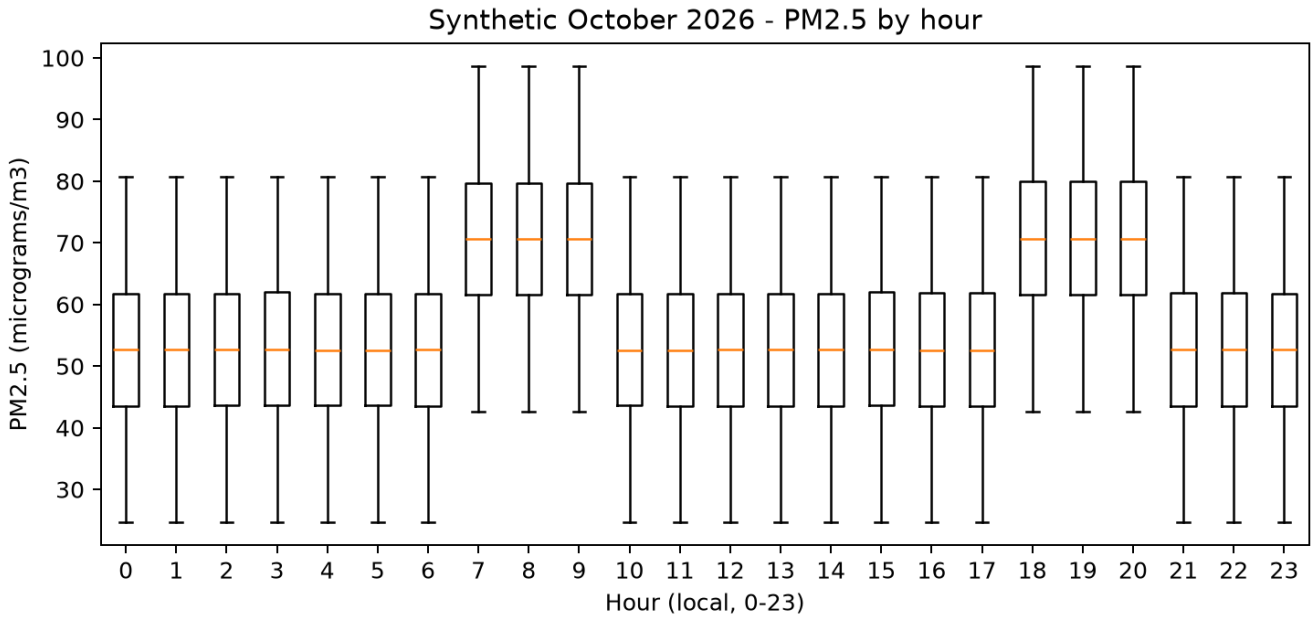
Adyar's daily mean rises from 33.57 on October 1 to 49.58 on October 31, with intermediate oscillations. Guindy follows a similar shape at a higher level because both share the same synthetic time pattern; this is evidence that the plotting pipeline works, not a discovery about the real city.



King's Park has the largest synthetic monthly mean (73.6489), and Adyar the smallest (40.6632). Every station has 740 valid readings, so coverage is equal in this fixture; a real comparison should inspect unequal coverage and missing-data patterns before ranking stations.



The scatter contains broad bands and substantial PM2.5 variation at similar temperatures. Both variables are generated from time-related formulas; the chart supports neither a simple causal relationship nor a claim that temperature alone predicts pollution.



The distributions at hours 7-9 and 18-20 are shifted upward relative to other hours, matching the generator's 18-unit addition. Overlapping boxes show substantial variation across dates and stations, so a worst-hour ranking of means does not describe every individual reading.

Alternative: csv loops versus pandas

The timed comparison isolates the exercise's PM2.5 column cleaning and min-max scaling on identical generated three-column files. Each file includes 10% NA, 10% blank, 10% negative, 10% above 999 and 60% valid readings; both methods agree on valid count, sum and normalized sum. Full date/unit/duplicate validation is outside this microbenchmark, so these times are not claimed as timings of every B1 rule.

The loop reads csv.DictReader rows, filters and stores floats, then scales them. pandas read_csv, replace, to_numeric, dropna, between and vectorised arithmetic perform the equivalent operations. Both are O(n) time and O(n) retained-data memory; pandas uses additional arrays/masks and DataFrame storage. Chunked pandas or streamed batches would

change the memory model, but were not benchmarked.

Rows	csv loop ms	pandas ms	Loop peak MiB	pandas peak MiB
1000	0.506	1.859	28.6	72.8
100000	46.622	24.672	32.6	89.6
1000000	471.518	230.022	75.9	208.5

The values are three-run medians in milliseconds; memory is the maximum process peak RSS across the runs in MiB. Each trial runs in a fresh process. Imports are excluded from elapsed time, while RSS includes interpreter and imported libraries; peak RSS is not just the DataFrame's byte size and is platform-specific. Files are likely warm in the operating-system cache. pandas is slower for 1,000 rows because its setup costs dominate, but approximately twice as fast at one million rows while using substantially more process memory. All raw runs are in cleaning_benchmarks.json.

Task B2 - Engineer adversarial data (5 marks)

File and SQL challenges

The files in TaskB2/inputs are deliberately constructed, then passed through the same clean_file entry point. The program raises SourceError with an explicit cause; the caller records the failure and can continue to a different source. A bad file is not reported as a successful zero-reading day.

Input	Observed outcome
empty.csv	expected exact comma-separated header: station,date,hour,pm25,pm10,no,temperature,temp_unit
latin1.csv	UnicodeDecodeError: 'utf-8' codec can't decode byte 0xe1 in position 56: invalid continuation byte
missing_column.csv	expected exact comma-separated header: station,date,hour,pm25,pm10,no,temperature,temp_unit
unclosed_quote.csv	Error: unexpected end of data
wrong_delimiter.csv	expected exact comma-separated header: station,date,hour,pm25,pm10,no,temperature,temp_unit

The missing-column and delimiter tests show why an explicit schema contract is necessary. The encoding test shows that decoding is part of acquisition, not an incidental display choice; a production adapter can accept a documented alternate encoding, but automatic guessing could corrupt names. The empty-file test distinguishes missing observations from actual zero pollution. The unclosed-quote test shows that structural CSV corruption is a file-level failure.

A legitimate station called King's Park illustrates the quoting problem. Concatenation builds WHERE station='King's Park', and SQLite raises OperationalError near s. A placeholder query with that exact station string returns 40.0. Parameter binding lets an apostrophe remain data.

For Adyar' OR '1'='1, string concatenation changes the WHERE clause into a tautology and the unsafe AVG query returns 61.0, the average of all three test stations (48,95,40). Binding the same text with ? searches for that complete literal name, which does not exist, so AVG returns SQL NULL, exposed as Python None. This is not an error and it is not zero. The result is one aggregate row, rather than every raw row; a SELECT * version of the unsafe query would reveal all matching raw rows.

The assignment's separate four-row Complete the Code program produces exactly:

```
54.0 108.0
None
```

Use placeholders for values, never string interpolation, even when an input looks like a station. Identifier choices such as table or column names require a fixed allowlist because ordinary value placeholders cannot substitute arbitrary SQL syntax. The queries here use fixed identifiers.

Sentiment adversaries

The four explicit probes are recorded in evidence.json. 'Good, another smoke-filled evening.' scores neutral although the intended sarcastic complaint is negative. 'The road is not not clean.' scores negative, missing the double-negation positive reading. 'Clean park, dirty road.' is neutral and agrees with the balanced mixed reference. 'Helpful staff, broken sensor; overall dissatisfied.' is neutral although the final opinion is negative. These cases expose missing discourse understanding, a one-token negation scope, and a limited lexicon; adding more positive words alone does not solve them.

Alternative: execute in a loop versus executemany

Both insert n rows and are O(n) time in the unindexed in-memory test; both use a single transaction and a commit after all rows. executemany reduces repeated Python-to-SQL call overhead. With maintained B-tree indexes, insertion can include O(log n) index work per row, and disk/transaction policy can dominate constants. A commit per row would be a different, unfair benchmark.

For 100,000 identical input rows, five-run medians were 33.409 ms for execute in a loop and 18.555 ms for executemany. Both databases were verified to have count 100,000 and PM2.5 sum 24,950,000. The prebuilt input list is O(n); a generator can feed either method with O(1) extra input state, apart from database/cache storage.

Debug-output experiment supporting A4

For 10,000 calls, median buffered print time was 1.367 ms, disabled debug logging 0.546 ms, and enabled buffered debug logging 21.919 ms. Enabled logging has record construction and formatting overhead; its benefit is controllability and structured severity, not a universal speed advantage. These timeit trials use StringIO and deliberately exclude terminal rendering and disk latency. Logs grow with output bytes; disabled logging avoids building the output buffer.

Task B3 - Loops versus libraries versus SQL (8 marks)

B3.1 Correctness and data generation

The generator creates 365 days x 24 hours for 1, 12 and 120 synthetic stations. Station S001 uses index 0. Its hourly PM2.5 formula is 20 + station_index + (day_index MOD 31) + hour/10. Therefore the independent expected daily mean is 20 + station_index + (day_index MOD 31) + 1.15, because the mean of hours 0..23 is 11.5.

The plain baseline collects one list per station/day and calls statistics.mean. The running alternative accumulates a sum and count per key. pandas reads the same CSV and uses groupby(['station','day']).pm25.mean(); SQLite reads a previously loaded database and uses SELECT station,day,AVG(pm25) FROM aq GROUP BY station,day. The small 8,760-row case compares every key/value pair between all four methods. At every size and every repetition, all returned means are also checked against the independent formula, and group counts must be stations x 365. All checks passed, including 43,800 groups at 1,051,200 rows.

B3.2 Measured time and memory

Stations	Rows	Lists ms	Running ms	pandas ms	SQL query ms	SQL load ms
1	8,760	6.841	4.974	2.840	1.513	10.154
12	105,120	80.788	59.698	15.891	20.251	84.240
120	1,051,200	810.374	610.721	150.563	225.650	849.702

Three independent process trials produce each method median. CSV methods include reading/parsing plus grouping; SQL query timing includes connection, query and result materialisation but excludes initial CSV loading. The separately shown SQL load is one measured parse/batch-insert/commit run, not a three-run median. Add it when comparing one-time end-to-end acquisition; at the largest size the load-plus-query cost is approximately 1.075 seconds. Pre-importing pandas excludes import startup from its time. These boundaries matter more than declaring a library universally fastest.

Rows	Lists MiB	Running MiB	pandas MiB	SQL MiB
8,760	28.8	28.6	78.7	29.6
105,120	33.6	29.8	92.0	34.2
1,051,200	86.8	48.7	198.8	46.9

Peak RSS includes runtime/import overhead and returned results. At 1,051,200 rows, the running dictionary used approximately 48.7 MiB versus 86.8 MiB for lists and 198.8 MiB for pandas. SQLite process RSS was 46.9 MiB, but its database and temporary storage consume disk space as well; RSS does not measure total storage.

B3.3 cProfile evidence

The largest plain-Python case produced 13,601,833 function calls in about 1.998 profiled seconds. The top-level mean_lists function has that entire cumulative time; among its hot children csv.DictReader.__next__ takes about 0.849 cumulative seconds and 0.498 internal seconds, while statistics.mean accounts for about 0.837 cumulative seconds. The CSV iterator is the largest internal-time function in this run. Exact-ratio and fraction work under statistics.mean contributes to the aggregation cost.

Profile instrumentation makes the run slower than the unprofiled 0.810-second median, so the profile time is diagnostic evidence, not another directly comparable benchmark. The full cumulative and internal rankings are saved in profile.txt. Replacing per-group lists/statistics.mean with sum/count avoids retaining every float and avoids some exact-arithmetic overhead, while tolerance-based comparison accounts for floating-point summation differences.

B3.4 Engineering choice and complexity

For a one-time analysis of a moderately sized CSV already fitting memory, pandas is concise and was fastest among these CSV methods. For an automated daily report, ingest each file once into the validated SQLite store and query only the relevant period; repeatedly reparsing the complete history wastes work and complicates consistency. On an environment without pandas or with a strict memory budget, the running dictionary is an effective standard-library fallback.

The list and running algorithms are both expected O(n) time. Lists store O(n) values plus O(g) keys; running sums/counts store O(g), where g = stations x observed days. pandas retains O(n) data plus grouping structures. SQL grouping has plan-dependent sort/index costs and can use temporary disk storage. For this fixed 24-readings-per-group generator, g = n/24, so both n and g grow with the station count: the memory improvement is a real constant reduction here, not evidence of constant memory as the network grows. If the number of observations per fixed group rises, the running-state advantage becomes stronger.

```
ALGORITHM RUNNING_MEANS
INPUT: CSV readings
OUTPUT: station/day means
BEGIN
    INITIALISE group dictionary
    FOR EACH reading DO
        SET key TO station and day
        INITIALISE sum/count for key if absent
        ADD PM2.5 TO sum
        INCREMENT count
    ENDFOR
    FOR EACH key DO
        OUTPUT key and sum / count
    ENDFOR
END
```

Task B4 - Final engineering decision (5 marks)

B4.1 Practical sentiment, summary and chatbot

TaskB4/complaints.json contains 50 distinct, multi-sentence synthetic complaints. Before running the scorer, an illustrative reference sentiment and salient-sentence position were specified for each. These are illustrative reference judgements, not an independently annotated benchmark. The student must perform the requested own-reading comparison; personal_review.csv provides blank columns for that work without inventing answers.

The implemented lexicon agrees with 31/50 illustrative labels (62%). Frequency-based one-sentence summaries select the nominated salient sentence in 11/50 cases (22%), compared with 7/50 (14%) for the first sentence. This strict exact-sentence measure penalises equally reasonable alternate sentences and is not a complete semantic quality metric. Even so, the low figures reveal that both baselines miss context and cannot be trusted as autonomous complaint triage.

The following table makes every comparison auditable. Key-sentence positions are one-based here; JSON stores zero-based indices. The full texts, predicted scores and both generated summaries are in evaluation.json.

ID	Reference	Prediction	Agree	Key sentence	Freq hit	First hit
1	positive	positive	Yes	2	No	No
2	positive	positive	Yes	2	No	No
3	positive	positive	Yes	2	No	No
4	positive	positive	Yes	2	No	No
5	positive	positive	Yes	1	Yes	Yes
6	negative	negative	Yes	2	No	No
7	negative	negative	Yes	2	No	No
8	negative	negative	Yes	2	No	No
9	negative	negative	Yes	2	No	No
10	negative	negative	Yes	2	No	No
11	negative	negative	Yes	1	Yes	Yes
12	negative	negative	Yes	2	No	No
13	negative	negative	Yes	1	Yes	Yes
14	negative	negative	Yes	2	No	No
15	negative	negative	Yes	2	No	No
16	positive	positive	Yes	1	Yes	Yes
17	positive	positive	Yes	1	Yes	Yes
18	positive	positive	Yes	1	Yes	Yes
19	positive	positive	Yes	2	No	No
20	positive	positive	Yes	2	No	No
21	neutral	neutral	Yes	2	No	No

ID	Reference	Prediction	Agree	Key sentence	Freq hit	First hit
22	neutral	neutral	Yes	2	No	No
23	neutral	neutral	Yes	2	No	No
24	neutral	neutral	Yes	2	No	No
25	neutral	neutral	Yes	2	No	No
26	negative	negative	Yes	2	Yes	No
27	negative	negative	Yes	2	Yes	No
28	negative	negative	Yes	2	Yes	No
29	negative	positive	No	2	No	No
30	negative	positive	No	1	No	Yes
31	positive	negative	No	2	No	No
32	negative	positive	No	2	No	No
33	positive	negative	No	2	Yes	No
34	positive	negative	No	2	No	No
35	negative	positive	No	2	No	No
36	negative	positive	No	3	No	No
37	positive	neutral	No	3	No	No
38	neutral	neutral	Yes	3	No	No
39	negative	neutral	No	3	No	No
40	positive	neutral	No	3	No	No
41	positive	neutral	No	2	No	No
42	negative	neutral	No	2	No	No
43	positive	neutral	No	2	No	No
44	negative	neutral	No	2	No	No
45	negative	neutral	No	2	No	No
46	negative	positive	No	2	No	No
47	negative	positive	No	2	Yes	No
48	positive	positive	Yes	2	No	No
49	positive	negative	No	2	No	No
50	positive	positive	Yes	2	No	No

For example, 'Such helpful staff, they ignored every call' can score positively because helpful is in the lexicon and ignored is not. 'The lane is not not clean' illustrates double-negation failure. 'The old service was bad but the current service is good' cancels to neutral, missing that the later clause and conclusion express satisfaction. The baseline's

three possible labels also cannot distinguish factual neutral text from a mixture of strong positive and negative aspects.

The completed exercise preserves its given tokenisation and produces:

```
positive | Roads are clean and the bus was fast.
negative | Smoke from the factory, air is bad.
negative | The staff were not helpful.
positive | Not bad, service improved.
```

The practical chatbot passes eight tests: valid Adyar, quoted station name, missing readings, two stations, unknown station, invalid calendar date, unsupported intent and absent date. A fixed test clock makes today mean 2026-10-01; the saved answer is a synthetic daily mean with its count, not a live reading. Its answer is checked against an independent sum/count query, and an extractive-summary check verifies that the returned sentence occurs in the original complaint.

B4.2 Index the database and measure the slowest query

The controlled test uses B3's 1,051,200-row unindexed aq table copied to a separate experiment database. It measures the three A1-style query shapes, records EXPLAIN QUERY PLAN, chooses the slowest baseline by its median, then adds CREATE INDEX idx_station_day ON aq(station,day). Seven warm-query trials are saved for each before/after case. Result equivalence is checked to a floating-point tolerance.

Query	Before ms	After ms	Before / after
station_month	19.9305	0.0283	703.41x
station_comparison	31.8970	28.8034	1.11x
worst_hours	32.0026	32.4056	0.99x

The baseline slowest query was worst_hours, at about 32.003 ms; after indexing it took 32.406 ms. That is effectively unchanged at this measurement precision, with a small run-to-run variation rather than a demonstrated improvement. It filters only on date and groups by hour, so a station-leading index does not provide a selective station range. The station/month query improved greatly because it can search station equality followed by a day interval; the full-city station comparison improves only modestly. Index construction took 0.196199 seconds, and indexes also occupy disk and add write cost.

This is deliberately a heap-table experiment. B1's PRIMARY KEY (station, day, hour) already creates an index with the required prefix; adding another (station, day) index there would normally duplicate useful access ordering. For the city-wide report, test a date-leading index such as (day, hour, pm25) or a maintained daily/hourly summary rather than assuming every index speeds every query. Those extra production candidates are recommendations for a follow-up workload test, not measured improvements in this report.

B4.3 Decision at the expanded scale

If only the 100 added stations report every 15 minutes while the original 12 stay hourly, the network produces 100 x 96 + 12 x 24 = 9,888 readings/day, or 3,609,120 in a 365-day year. If all 112 stations switch to 15-minute intervals, it produces 10,752/day and 3,924,480/year. These are around 3.4-3.7 times the largest benchmark, but its timings should not be extrapolated as guaranteed latency: cache, disk, indexes and concurrency change.

Keep immutable raw CSVs plus checksum/provenance, stage and validate each incoming file, record rejected rows, then insert bounded batches transactionally. Replace the hour-only key with an unambiguous timestamp or day/minute slot; the existing hourly schema would wrongly collapse four quarter-hour readings. Track station timezone and schedule, distinguish late corrections from duplicates, and publish per-column coverage and quality flags. Apply validation before accepting data, and fit scaling parameters only on the relevant training/reference period.

SQLite is a practical starting point for this classroom-sized single-writer service and scheduled reports; monitor lock contention, backups, file growth and query latency. If concurrent writers or availability requirements grow, migrate the same validated schema to a server database. Query date ranges, retain station/time indexing, and maintain summary sum/count/min/max so repeated reports do not scan all history. Use pandas on selected extracts for exploratory analysis and matplotlib for labelled reports. The million-row measurements support this workflow, not an untested production

capacity guarantee.

The 50-case medians were 0.1611 ms for sentiment, 0.2810 ms for frequency summaries and 0.0448 ms for first-sentence summaries. These short English fixtures show low computational cost; 50,000 monthly complaints are primarily a quality and oversight problem for this baseline. Do not route or dismiss complaints solely from a sentiment sign. Keep the simple scorer as an explainable demonstration, collect representative independently reviewed multilingual labels, evaluate a trained baseline on a held-out test set, and retain human review for uncertain or important cases.

Alternative: first k sentences

If sentences are already stored in an array, selecting a fixed k sentence references is O(k), conventionally O(1) for fixed k. Producing the copied output still costs its character length. Starting from raw text, this implementation splits the entire complaint before slicing, so first_k is O(W) time/space in input words or characters, not O(1) end-to-end. A streaming implementation could stop after the kth sentence, costing the prefix length.

Frequency counting and sentence scoring cost O(W); the implemented sorting of S sentence scores adds O(S log S), and retained tokens/frequencies use O(W + V + S) space. A top-k heap could reduce selection to O(S log k), but is not used here. On the 50-complaint rubric, frequency improves exact salience matches by four cases, yet both methods remain poor: first sentences often introduce the topic while the actionable detail or overall judgement comes later. Repetition can also mislead the frequency method. The full paired summaries support inspection instead of a claim that frequency always gives better summaries.

References and final checks

The supplied assignment defines the tasks and marking scheme. Python's sqlite3 documentation describes placeholder binding and executemany: https://docs.python.org/3/library/sqlite3.html. SQLite's Query Planning guide explains index search, multi-column ordering and matching-row work: https://www.sqlite.org/queryplanner.html. pandas documents the split-apply-combine groupby model: https://pandas.pydata.org/docs/user_guide/groupby.html. Library links were checked on 5 November 2026; the executed package versions are listed above and may differ from the latest documentation.

All expected Complete the Code outputs match. The small cleaning fixture, malformed-file checks, SQL injection and quote tests, insertion sums/counts, all-size mean comparisons, eight chatbot cases and index result equivalence passed. Timings, profile and images were generated from actual runs. Remaining student actions are personal execution/screenshots, their own 50-complaint review, conformity with any separately supplied pseudocode template, and sharing/submitting the repository to the faculty. These steps remain for submission.

Appendix - Active Python source

Each listing is labelled with its task folder and filename. Save related files in the shown folders, or run the files in the companion ZIP. It also contains the 50 complaints, source CSVs, SQLite database, raw measurements and output transcripts. Long code lines are visually wrapped in this PDF; use the original Python files for execution.

TaskB1/complete_code.py

```
import csv
import io
RAW = """station,date,pm25
Adyar,2026-10-01,48
Adyar,2026-10-02,NA
Adyar,2026-10-03,
Guindy,2026-10-01,95
Guindy,2026-10-02,-5
Guindy,2026-10-03,121"""
def load_clean(text):
    """Returns (station, date, pm25) tuples for valid, non-negative readings only."""
    rows = []
    for row in csv.DictReader(io.StringIO(text)):
        value = row['pm25'].strip()
        if value in ('', 'NA'):
            continue
        pm = float(value)
        if 0 <= pm <= 999:
            rows.append((row['station'],row['date'],pm))
    return rows

def min_max(values):
    if not values:
        return []
    low, high = min(values), max(values)
    if high == low:
        return [0.0]*len(values)
    return [round((v-low)/(high-low),3) for v in values]

clean = load_clean(RAW)
print(clean)
print(min_max([pm for _, _, pm in clean]))
# Additional check requested after the original program.
assert load_clean(RAW+'\nAdyar,2026-10-04,1000') == clean
print('Above-999 row skipped: PASS')
```

TaskB1/demo.py

```
import csv
import json
import math
import statistics
from pathlib import Path
from pipeline import FIELDS, STATIONS, UNITS, clean_file, min_max, z_scores, store, queries
ROOT=Path(__file__).resolve().parent

def write_csv(path, rows):
    with path.open('w',encoding='utf-8',newline='') as f:
        w=csv.writer(f);w.writerow(FIELDS);w.writerows(rows)

def main():
    base=['Adyar','2026-10-01',0,48,80,15,30,'C']
    rows=[base, ['  aDyAr  ','2026-10-01',1,60,90,20,31,' c '],
          ['Guindy','2026-10-01',0,95,150,25,86,'F'],
          ["King's Park",'2026-10-01',0,40,70,10,29,'C']]
    # Each row independently illustrates one declared quality problem.
    changes=[(3,''),(4,'NA'),(5,-1),(3,1000),(3,'broken'),(3,'nan'),
             (3,'inf'),(6,61),(1,'2026-02-30'),(1,'01/10/2026'),
             (2,24),(2,'1.5'),(0,'Unknown'),(7,'F')]
    for index,value in changes:
        row=base.copy();row[index]=value;rows.append(row)
    rows.extend([base,base[:-1],base+['extra']])
    write_csv(ROOT/'small_messy.csv',rows)
    clean,issues=clean_file(ROOT/'small_messy.csv')
    assert len(clean)==4 and len(issues)==17
    assert clean[1][:3]==('Adyar','2026-10-01',1)
    assert clean[2][-1]==30.0
    assert min_max([5,5])==[0,0] and z_scores([5,5])==[0,0]
    assert min_max([])==[] and z_scores([])==[]
    assert min_max([48,60,95,40])==[(x-40)/55 for x in [48,60,95,40]]
    assert abs(statistics.fmean(z_scores([48,60,95,40])))<1e-12
    (ROOT/'small_before_after.json').write_text(json.dumps({'raw_rows':rows,'clean_rows':clean,'iss
        ues':issues,'pm25_minmax':min_max([r[3] for r in clean]),'pm25_zscore':z_scores([r[3] for r
         in clean])},indent=2))
    with (ROOT/'small_cleaned.csv').open('w',newline='') as f:
        w=csv.writer(f);w.writerow(['station','day','hour','pm25','pm10','no','temperature_c']);w.w
            riterows(clean)
    monthly=[]
    for i,station in enumerate(STATIONS):
        for day in range(1,32):
            for hour in range(24):
                pm=round(25+i*3+0.7*day+(18 if hour in {7,8,9,18,19,20} else 0)+4*math.sin(day),2)
                temp=round(25+5*math.sin((hour-6)*math.pi/12)+0.05*day,2)
                reading=[station,f'2026-10-
                    {day:02}',hour,pm,round(pm*1.5,2),round(10+i+hour/5,2),round(temp*9/5+32,2) if
                    UNITS[station]=='F' else temp,UNITS[station]]
                if (day*24+hour+i)%173==0: reading[3]='NA'
                monthly.append(reading)
    write_csv(ROOT/'october_synthetic.csv',monthly)
    clean,issues=clean_file(ROOT/'october_synthetic.csv')
    store(clean,ROOT/'air_quality.sqlite')
    result=queries(ROOT/'air_quality.sqlite')
    (ROOT/'query_results.json').write_text(json.dumps(result,indent=2))
    (ROOT/'monthly_audit.json').write_text(json.dumps({'input_rows':len(monthly),'accepted':len(cle
        an),'rejected':len(issues),'issues':issues},indent=2))
    import sqlite3
    import pandas as pd
    import os, tempfile
    os.environ.setdefault("MPLCONFIGDIR", str(Path(tempfile.gettempdir())/"csa08-mpl"))
    import matplotlib
    matplotlib.use('Agg')
    import matplotlib.pyplot as plt
    with sqlite3.connect(ROOT/'air_quality.sqlite') as con:
        daily=pd.read_sql_query('SELECT * FROM daily_summary ORDER BY station,day',con)
        frame=pd.read_sql_query('SELECT * FROM readings',con)
    daily.to_csv(ROOT/'daily_summary.csv',index=False,float_format='%.4f')
    plt.rcParams.update({'font.size':10,'figure.dpi':130})
    def save(name):
        plt.tight_layout();plt.savefig(ROOT/name,dpi=180);plt.close()
    fig,ax=plt.subplots(figsize=(8.2,4))
    for station in ['Adyar','Guindy']:
        d=daily[daily.station==station];ax.plot(pd.to_datetime(d.day),d['mean'],label=station)
    ax.set(title='Synthetic October 2026 - daily PM2.5 trend',xlabel='Date',ylabel='Mean PM2.5
        (micrograms/m3)');ax.legend();fig.autofmt_xdate();save('trend.png')
    fig,ax=plt.subplots(figsize=(8.2,4.5))
    frame.groupby('station').pm25.mean().sort_values().plot.barh(ax=ax,color='#1c7293')
```

```
    ax.set(title='Synthetic October 2026 - station comparison',xlabel='Mean PM2.5
        (micrograms/m3)',ylabel='Station');save('stations.png')
    fig,ax=plt.subplots(figsize=(8.2,4))
    d=frame[frame.station=='Adyar'];ax.scatter(d.temperature_c,d.pm25,s=8,alpha=.35)
    ax.set(title='Synthetic Adyar - temperature and PM2.5',xlabel='Temperature (degrees
        C)',ylabel='PM2.5 (micrograms/m3)');save('scatter.png')
    fig,ax=plt.subplots(figsize=(8.2,4))
    ax.boxplot([frame[frame.hour==h].pm25.to_numpy() for h in
        range(24)],tick_labels=list(range(24)),showfliers=False)
    ax.set(title='Synthetic October 2026 - PM2.5 by hour',xlabel='Hour (local, 0-23)',ylabel='PM2.5
         (micrograms/m3)');save('hours.png')
    print('Small-file validation: 4 accepted, 17 rejected; canonical names/Fahrenheit/normalisation
         PASS')
    print(f'Month: {len(monthly)} input; {len(clean)} accepted; {len(issues)} rejected')
    print(json.dumps(result,indent=2))
    print('Four charts and daily_summary.csv generated; data are synthetic.')
if __name__=='__main__': main()
```

TaskB1/pipeline.py

```
"""Validated CSV -> SQLite pipeline; all demonstration data are synthetic."""
import csv
import json
import logging
import math
import sqlite3
import statistics
from datetime import date
from pathlib import Path

LOG = logging.getLogger(__name__)
STATIONS = ['Adyar', 'Guindy', 'Velachery', 'T Nagar', 'Anna Nagar',
            'Ambattur', 'Perambur', 'Royapuram', 'Mylapore',
            'Tambaram', 'Porur', "King's Park"]
UNITS = {s: ('F' if s in {'Guindy', 'Ambattur'} else 'C') for s in STATIONS}
LOOKUP = {' '.join(s.split()).casefold(): s for s in STATIONS}
FIELDS = ['station', 'date', 'hour', 'pm25', 'pm10', 'no', 'temperature', 'temp_unit']

class SourceError(ValueError):
    """A file cannot be interpreted under the declared source contract."""

def min_max(values):
    if not values:
        return []
    lo, hi = min(values), max(values)
    return [0.0 if hi == lo else (v-lo)/(hi-lo) for v in values]

def z_scores(values):
    if not values:
        return []
    mu = statistics.fmean(values)
    sigma = statistics.pstdev(values)
    return [0.0 if sigma == 0 else (v-mu)/sigma for v in values]

def number(text, label):
    if text is None or text.strip().casefold() in {'', 'na'}:
        raise ValueError(f'missing {label}')
    value = float(text)
    if not math.isfinite(value):
        raise ValueError(f'non-finite {label}')
    return value

def clean_file(path):
    """Drop incomplete/invalid rows; retain first valid duplicate key."""
    accepted, issues, seen = [], [], set()
    try:
        with Path(path).open(encoding='utf-8-sig', newline='') as stream:
            reader = csv.DictReader(stream, strict=True)
            if reader.fieldnames != FIELDS:
                raise SourceError('expected exact comma-separated header: '+','.join(FIELDS))
            for raw in reader:
                line = reader.line_num
                try:
                    if None in raw or any(v is None for v in raw.values()):
                        raise ValueError('wrong number of columns')
                    station = LOOKUP.get(' '.join(raw['station'].split()).casefold())
                    if station is None:
                        raise ValueError('unknown or empty station')
                    day_text = raw['date'].strip()
                    day = date.fromisoformat(day_text).isoformat()
                    if day != day_text:
                        raise ValueError('date must be YYYY-MM-DD')
                    hour_text = raw['hour'].strip()
                    if not hour_text.isascii() or not hour_text.isdigit():
                        raise ValueError('hour must be an integer')
                    hour = int(hour_text)
                    if not 0 <= hour <= 23:
                        raise ValueError('hour outside 0..23')
                    pm25 = number(raw['pm25'], 'pm25')
                    pm10 = number(raw['pm10'], 'pm10')
                    no = number(raw['no'], 'no')
                    temperature = number(raw['temperature'], 'temperature')
                    unit = raw['temp_unit'].strip().upper()
                    if unit != UNITS[station]:
                        raise ValueError('temperature unit disagrees with station metadata')
                    if unit == 'F':
                        temperature = (temperature-32)*5/9
                    for label, value, upper in [('pm25',pm25,999), ('pm10',pm10,1999),
```

```
                    ('no',no,999)]:
                        if not 0 <= value <= upper:
                            raise ValueError(f'{label} outside 0..{upper}')
                    if not -50 <= temperature <= 60:
                        raise ValueError('temperature outside -50..60 C')
                    key = (station, day, hour)
                    if key in seen:
                        raise ValueError('duplicate station/day/hour; first valid row retained')
                    seen.add(key)
                    accepted.append((station, day, hour, pm25, pm10, no, temperature))
                except (ValueError, OverflowError) as exc:
                    issues.append({'line':line, 'reason':str(exc), 'raw':raw})
        LOG.info('Accepted %d; rejected %d from %s',len(accepted),len(issues),path)
        return accepted, issues
    except (OSError, UnicodeError, csv.Error) as exc:
        raise SourceError(f'{type(exc).__name__}: {exc}') from exc

SCHEMA = """
CREATE TABLE stations (
    station TEXT PRIMARY KEY,
    temperature_unit TEXT NOT NULL CHECK (temperature_unit IN ('C','F'))
);
CREATE TABLE readings (
    station TEXT NOT NULL REFERENCES stations(station),
    day TEXT NOT NULL,
    hour INTEGER NOT NULL CHECK (hour BETWEEN 0 AND 23),
    pm25 REAL NOT NULL CHECK (pm25 BETWEEN 0 AND 999),
    pm10 REAL NOT NULL CHECK (pm10 BETWEEN 0 AND 1999),
    no REAL NOT NULL CHECK (no BETWEEN 0 AND 999),
    temperature_c REAL NOT NULL CHECK (temperature_c BETWEEN -50 AND 60),
    PRIMARY KEY (station, day, hour)
);
CREATE VIEW daily_summary AS
SELECT station, day, MIN(pm25) AS minimum, MAX(pm25) AS maximum,
       AVG(pm25) AS mean, COUNT(*) AS valid_count
FROM readings GROUP BY station, day;
"""

def store(rows, db):
    """Build a fresh demonstration DB; caller chooses its output path."""
    with sqlite3.connect(db) as con:
        con.execute('PRAGMA foreign_keys=ON')
        con.executescript('DROP VIEW IF EXISTS daily_summary; DROP TABLE IF EXISTS readings; DROP
            TABLE IF EXISTS stations;'+SCHEMA)
        con.executemany('INSERT INTO stations VALUES (?, ?)', UNITS.items())
        con.executemany('INSERT INTO readings VALUES (?, ?, ?, ?, ?, ?, ?)', rows)
        con.commit()

QUERIES = {
    'adyar_october_average': ("""SELECT AVG(pm25), COUNT(*) FROM readings
        WHERE station=? AND day>=? AND day<?""", ('Adyar','2026-10-01','2026-11-01')),
    'station_comparison': ("""SELECT station, AVG(pm25), COUNT(*) FROM readings
        WHERE day>=? AND day<? GROUP BY station ORDER BY AVG(pm25) DESC""",
            ('2026-10-01','2026-11-01')),
    'worst_hours': ("""SELECT hour, AVG(pm25), COUNT(*) FROM readings
        WHERE day>=? AND day<? GROUP BY hour ORDER BY AVG(pm25) DESC, hour LIMIT 5""",
            ('2026-10-01','2026-11-01')),
}

def queries(db):
    with sqlite3.connect(db) as con:
        return {name: con.execute(sql,params).fetchall() for name,(sql,params) in QUERIES.items()}

if __name__ == '__main__':
    import argparse
    parser=argparse.ArgumentParser()
    parser.add_argument('input',type=Path)
    parser.add_argument('database',type=Path)
    args=parser.parse_args()
    rows, issues=clean_file(args.input)
    store(rows,args.database)
    print(json.dumps({'accepted':len(rows),'rejected':len(issues),'queries':queries(args.database)}
        ,indent=2))
```

TaskB2/adversarial.py

```
import csv
import io
import json
import logging
import sqlite3
import statistics
import sys
import time
import timeit
from pathlib import Path
ROOT=Path(__file__).resolve().parent
sys.path.insert(0,str(ROOT.parent/'TaskB1'))
sys.path.insert(0,str(ROOT.parent/'TaskB4'))
from pipeline import FIELDS, SourceError,clean_file
from ai import label

def main():
    inputs=ROOT/'inputs';inputs.mkdir(exist_ok=True)
    header=','.join(FIELDS)
    (inputs/'missing_column.csv').write_text('station,date,hour,pm25\nAdyar,2026-10-01,0,48\n')
    (inputs/'wrong_delimiter.csv').write_text(header.replace(',',';')+'\nAdyar;2026-10-
        01;0;48;80;15;30;C\n')
    (inputs/'empty.csv').write_text('')
    (inputs/'latin1.csv').write_bytes((header+'\nAdyár,2026-10-
        01,0,48,80,15,30,C\n').encode('latin-1'))
    (inputs/'unclosed_quote.csv').write_text(header+'\n"Adyar,2026-10-01,0,48,80,15,30,C\n')
    cases=[]
    for p in sorted(inputs.glob('*.csv')):
        try:clean_file(p)
        except SourceError as
            exc:cases.append({'file':p.name,'outcome':type(exc).__name__,'message':str(exc)})
        else:raise AssertionError(f'Expected rejection: {p.name}')
    con=sqlite3.connect(':memory:')
    con.execute('CREATE TABLE aq(station TEXT,pm25 REAL)')
    con.executemany('INSERT INTO aq VALUES (?,?)',[('Adyar',48),('Guindy',95),("King's Park",40)])
    sql=[]
    for name in ["King's Park","Adyar' OR '1'='1"]:
        unsafe="SELECT AVG(pm25) FROM aq WHERE station='"+name+"'"
        try:out=con.execute(unsafe).fetchone()[0]
        except sqlite3.Error as exc:out=type(exc).__name__+': '+str(exc)
        safe=con.execute('SELECT AVG(pm25) FROM aq WHERE station=?',(name,)).fetchone()[0]

            sql.append({'input':name,'unsafe_query':unsafe,'unsafe_result':out,'parameterized_resul
            t':safe})
    assert sql[0]['parameterized_result']==40
    assert isinstance(sql[0]['unsafe_result'],str)
    assert abs(sql[1]['unsafe_result']-61)<1e-12 and sql[1]['parameterized_result'] is None
    con.close()
    timings={}
    data=[('Adyar',float(i%500)) for i in range(100000)]
    for method in ['execute_loop','executemany']:
        runs=[]
        for _ in range(5):
            with sqlite3.connect(':memory:') as con:
                con.execute('CREATE TABLE aq(station TEXT,pm25 REAL)')
                start=time.perf_counter()
                if method=='execute_loop':
                    for row in data:con.execute('INSERT INTO aq VALUES (?,?)',row)
                else:con.executemany('INSERT INTO aq VALUES (?,?)',data)
                con.commit();runs.append(time.perf_counter()-start)
                assert con.execute('SELECT COUNT(*),SUM(pm25) FROM
                    aq').fetchone()==(100000,24950000)
        timings[method]={'runs_seconds':runs,'median_seconds':statistics.median(runs)}
    probes=[('Good, another smoke-filled evening.','negative'),('The road is not not
        clean.','positive'),
            ('Clean park, dirty road.','neutral'),('Helpful staff, broken sensor; overall
                dissatisfied.','negative')]
    sentiment=[{'text':t,'reference':ref,'actual':label(t),'agrees':ref==label(t)} for t,ref in
        probes]
    logger=logging.getLogger('experiment');logger.propagate=False
    stream=io.StringIO();logger.handlers[:]=[logging.StreamHandler(stream)]
    # Buffered output excludes terminal rendering. Exactly 10,000 calls per trial.
    debug={}
    for method in ['print_buffered','logging_disabled','logging_enabled']:
        runs=[]
        for _ in range(3):
            buffer=io.StringIO();stream.seek(0);stream.truncate(0)
            logger.setLevel(logging.WARNING if method=='logging_disabled' else logging.DEBUG)
```

```
            action=(lambda:print('row',42,file=buffer)) if method=='print_buffered' else
                (lambda:logger.debug('row %s',42))
            runs.append(timeit.timeit(action,number=10000))
        debug[method]={'runs_seconds':runs,'median_seconds':statistics.median(runs)}
    evidence={'files':cases,'sql':sql,'insert_100000_rows_one_transaction':timings,'sentiment':sent
        iment,'debug_10000_calls':debug}
    (ROOT/'evidence.json').write_text(json.dumps(evidence,indent=2))
    print(json.dumps(evidence,indent=2))
    print('Adversarial checks and insertion equivalence: PASS')
if __name__=='__main__':main()
```

TaskB2/complete_code.py

```
import sqlite3
readings = [("Adyar", "2026-10-01", 48.0), ("Guindy", "2026-10-01", 95.0),
            ("Guindy", "2026-10-03", 121.0), ("Adyar", "2026-10-04", 60.0)]
con = sqlite3.connect(":memory:")
con.execute("CREATE TABLE aq (station TEXT, day TEXT, pm25 REAL)")
con.executemany('INSERT INTO aq VALUES (?,?,?)',readings)
con.commit()
def station_average(name):
    return con.execute('SELECT AVG(pm25) FROM aq WHERE station = ?',(name,)).fetchone()[0]

print(station_average("Adyar"), station_average("Guindy"))
print(station_average("Adyar' OR '1'='1"))      # an injection attempt finds nothing
con.close()
```

TaskB3/benchmarks.py

```
"""Reproducible benchmarks. Generated large files stay outside the submission."""
import argparse
import csv
import cProfile
import io
import json
import math
import os
import pstats
import resource
import sqlite3
import statistics
import subprocess
import sys
import time
from collections import defaultdict
from datetime import date, timedelta
from pathlib import Path
ROOT=Path(__file__).resolve().parent

def rss_mib():
    raw=resource.getrusage(resource.RUSAGE_SELF).ru_maxrss
    return raw/(1024*1024 if sys.platform=='darwin' else 1024)

def clean_loops(path):
    values=[]
    with open(path,newline='') as f:
        for row in csv.DictReader(f):
            text=row['pm25']
            if text in ('','NA'):
                continue
            value=float(text)
            if math.isfinite(value) and 0<=value<=999:
                values.append(value)
    lo,hi=min(values),max(values)
    normalized=[0.0 if lo==hi else (x-lo)/(hi-lo) for x in values]
    return len(values),sum(values),sum(normalized)

def clean_pandas(path):
    import pandas as pd
    df=pd.read_csv(path,keep_default_na=False)
    df['pm25']=pd.to_numeric(df.pm25.replace({'NA':None,'':None}),errors='coerce')
    df=df.dropna(subset=['pm25'])
    df=df[df.pm25.between(0,999)].copy()
    lo,hi=df.pm25.min(),df.pm25.max()
    df['normalized']=0.0 if lo==hi else (df.pm25-lo)/(hi-lo)
    return len(df),float(df.pm25.sum()),float(df.normalized.sum())

def mean_lists(path):
    groups=defaultdict(list)
    with open(path,newline='') as f:
        for row in csv.DictReader(f):
            groups[(row['station'],row['day'])].append(float(row['pm25']))
    return {key:statistics.mean(values) for key,values in groups.items()}

def mean_running(path):
    groups={}
    with open(path,newline='') as f:
        for row in csv.DictReader(f):
            key=(row['station'],row['day'])
            if key not in groups: groups[key]=[0.0,0]
            groups[key][0]+=float(row['pm25']);groups[key][1]+=1
    return {key:total/count for key,(total,count) in groups.items()}

def mean_pandas(path):
    import pandas as pd
    frame=pd.read_csv(path)
    return frame.groupby(['station','day']).pm25.mean().to_dict()

def load_database(path,db):
    with sqlite3.connect(db) as con:
        con.execute('DROP TABLE IF EXISTS aq')
        con.execute('CREATE TABLE aq (station TEXT, day TEXT, hour INTEGER, pm25 REAL)')
        with open(path,newline='') as f:
            con.executemany('INSERT INTO aq VALUES (?,?,?,?)',
                ((r['station'],r['day'],int(r['hour']),float(r['pm25'])) for r in
                    csv.DictReader(f)))
        con.commit()
```

```
def mean_sql(db):
    with sqlite3.connect(db) as con:
        return {(s,d):m for s,d,m in con.execute('SELECT station,day,AVG(pm25) FROM aq GROUP BY
            station,day')}

def same(a,b):
    assert a.keys()==b.keys()
    assert all(math.isclose(a[k],b[k],rel_tol=1e-10,abs_tol=1e-9) for k in a)

def worker(kind,path):
    # Imports excluded from timing; RSS includes interpreter/import overhead.
    if 'pandas' in kind:
        import pandas
    fn={'clean_loops':clean_loops,'clean_pandas':clean_pandas,
        'lists':mean_lists,'running':mean_running,'pandas':mean_pandas,'sql':mean_sql}[kind]
    start=time.perf_counter();out=fn(path);elapsed=time.perf_counter()-start
    if isinstance(out,dict):
        # Every output group is validated, not just a summary checksum.
        for (station,day),value in out.items():
            si=int(station[1:])-1;di=(date.fromisoformat(day)-date(2026,1,1)).days
            expected=20+si+(di%31)+11.5/10
            assert math.isclose(value,expected,rel_tol=1e-10,abs_tol=1e-9)
        check={'groups':len(out),'sum_means':sum(out.values())}
    else:
        check={'valid_rows':out[0],'sum_pm25':out[1],'sum_normalized':out[2]}
    return {'seconds':elapsed,'process_peak_rss_mib':rss_mib(),'check':check}

def samples(kind,path):
    runs=[]
    for _ in range(3):
        result=subprocess.run([sys.executable,__file__,'--
            worker',kind,str(path)],text=True,capture_output=True,check=True)
        runs.append(json.loads(result.stdout))
    return {'runs':runs,'median_seconds':statistics.median(r['seconds'] for r in runs),
            'max_peak_rss_mib':max(r['process_peak_rss_mib'] for r in runs)}

def main(work):
    work.mkdir(parents=True,exist_ok=True)
    b1=[]
    for n in [1000,100000,1000000]:
        path=work/f'clean_{n}.csv'
        valid=[]
        with path.open('w',newline='') as f:
            w=csv.writer(f);w.writerow(['station','day','pm25'])
            for i in range(n):
                v='NA' if i%10==0 else '' if i%10==1 else -5 if i%10==2 else 1000 if i%10==3 else
                    i%500
                w.writerow(['Adyar','2026-10-01',v])
                if isinstance(v,int) and 0<=v<=999:valid.append(v)
        lo,hi=min(valid),max(valid)
        expected=[len(valid),sum(valid),sum((v-lo)/(hi-lo) for v in valid)]
        # Data repeat every 500 rows; first 1,000 contain the full range.
        methods={name:samples(name,path) for name in ['clean_loops','clean_pandas']}
        for method in methods.values():
            for run in method['runs']:
                check=list(run['check'].values())
                assert all(math.isclose(a,b,rel_tol=1e-9,abs_tol=1e-8) for a,b in
                    zip(check,expected))
        b1.append({'rows':n,'methods':methods})
        print(f'Cleaning {n}: '+str({k:round(v['median_seconds'],4) for k,v in
            methods.items()}),flush=True)
    (ROOT.parent/'TaskB1/cleaning_benchmarks.json').write_text(json.dumps(b1,indent=2))
    b3=[]
    for stations in [1,12,120]:
        path=work/f'year_{stations}.csv';db=work/f'year_{stations}.sqlite'
        with path.open('w',newline='') as f:
            w=csv.writer(f);w.writerow(['station','day','hour','pm25'])
            for si in range(stations):
                for di in range(365):
                    for hour in range(24):
                        w.writerow([f'S{si+1:03}',(date(2026,1,1)+timedelta(days=di)).isoformat(),h
                    our,round(20+si+di%31+hour/10,1)])
        start=time.perf_counter();load_database(path,db);load_seconds=time.perf_counter()-start
        methods={name:samples(name,db if name=='sql' else path) for name in
            ['lists','running','pandas','sql']}
        for method in methods.values():
            for run in method['runs']:assert run['check']['groups']==stations*365
        if stations==1:
            reference=mean_lists(path)
            for fn,arg in
                [(mean_running,path),(mean_pandas,path),(mean_sql,db)]:same(reference,fn(arg))
```

```
        b3.append({'stations':stations,'rows':stations*8760,'sql_load_seconds':load_seconds,'method
            s':methods})
        print(f'Year {stations} stations: '+str({k:round(v['median_seconds'],4) for k,v in
            methods.items()}),flush=True)
    (ROOT/'measurements.json').write_text(json.dumps(b3,indent=2))
    profile=cProfile.Profile();profile.runcall(mean_lists,work/'year_120.csv')
    stream=io.StringIO();pstats.Stats(profile,stream=stream).strip_dirs().sort_stats('cumulative').
        print_stats(30)
    stream.write('\nSORTED BY INTERNAL
        TIME\n');pstats.Stats(profile,stream=stream).strip_dirs().sort_stats('tottime').print_stats
        (20)
    (ROOT/'profile.txt').write_text(stream.getvalue())
    print('All four methods agree with independently calculated means at all three sizes. Profile
        saved.')

if __name__=='__main__':
    parser=argparse.ArgumentParser();parser.add_argument('--work-
        dir',type=Path);parser.add_argument('--worker',nargs=2)
    args=parser.parse_args()
    if args.worker:print(json.dumps(worker(*args.worker)))
    elif args.work_dir:main(args.work_dir)
    else:parser.error('--work-dir is required for a benchmark run')
```

TaskB4/ai.py

```
"""Transparent teaching baselines; not a validated public-service AI system."""
import re
import sqlite3
from collections import Counter
from datetime import date
POSITIVE={'good','clean','fast','helpful','improved'}
NEGATIVE={'bad','dirty','slow','rude','smoke','broken'}
NEGATORS={'not','never','no'}
STOP=set('a an the and or but is are was were be been being i we you they it this that to of in on
    at for from with as my our your has have had do does did please very there near every now today
     yesterday still also'.split())

def words(text):
    return re.findall(r"[a-z]+(?:'[a-z]+)?",text.lower())

def sentences(text):
    return [s.strip() for s in re.split(r'(?<=[.!?])\s+',text.strip()) if s.strip()]

def score(text):
    total=0
    for sentence in sentences(text):
        tokens=words(sentence)
        for i,word in enumerate(tokens):
            value=1 if word in POSITIVE else -1 if word in NEGATIVE else 0
            if i and tokens[i-1] in NEGATORS:value=-value
            total+=value
    return total

def label(text):
    value=score(text)
    return 'positive' if value>0 else 'negative' if value<0 else 'neutral'

def summary_indices(text,k=1):
    parts=sentences(text)
    if k<=0:return []
    token_lists=[[w for w in words(s) if w not in STOP] for s in parts]
    counts=Counter(w for tokens in token_lists for w in tokens)
    # Average frequency avoids rewarding a sentence only for being long.
    scores=[sum(counts[w] for w in tokens)/len(tokens) if tokens else 0 for tokens in token_lists]
    selected=sorted(range(len(parts)),key=lambda i:(-scores[i],i))[:k]
    return sorted(selected)

def summarize(text,k=1):
    parts=sentences(text)
    return ' '.join(parts[i] for i in summary_indices(text,k))

def first_k(text,k=1):
    return ' '.join(sentences(text)[:max(0,k)])

def chatbot(question,db,today=None):
    q=' '.join(question.lower().split())
    if 'pm2.5' not in q and 'pm25' not in q:
        return 'Ask for PM2.5, a station name, and today or a YYYY-MM-DD date.'
    with sqlite3.connect(db) as con:
        stations=[r[0] for r in con.execute('SELECT station FROM stations')]
        matched=[s for s in stations if re.search(r'(?<!\w)'+re.escape(s.lower())+r'(?!\w)',q)]
        if len(matched)!=1:return 'Please specify exactly one known station.'
        literal=re.findall(r'\b\d{4}-\d{2}-\d{2}\b',q)
        if len(literal)>1:return 'Please specify one date.'
        if literal:
            try:day=date.fromisoformat(literal[0]).isoformat()
            except ValueError:return 'Invalid date; use YYYY-MM-DD.'
        elif 'today' in words(q):day=(today or date.today()).isoformat()
        else:return 'Please specify today or a YYYY-MM-DD date.'
        mean,count=con.execute('SELECT AVG(pm25),COUNT(*) FROM readings WHERE station=? AND
            day=?',(matched[0],day)).fetchone()
    if not count:return f'No valid readings for {matched[0]} on {day}.'
    return f'{matched[0]} on {day}: mean PM2.5 {mean:.2f} micrograms/m3 from {count} valid
        readings.'
```

TaskB4/chatbot_demo.py

```
import json
from datetime import date
from pathlib import Path
from ai import chatbot,summarize,sentences
ROOT=Path(__file__).resolve().parent
DB=ROOT.parent/'TaskB1/air_quality.sqlite'
cases=[
 ('What is the PM2.5 in Adyar today?', 'Adyar on 2026-10-01:'),
 ("What is the PM2.5 in King's Park on 2026-10-01?", "King's Park on 2026-10-01:"),
 ('PM2.5 Adyar on 2026-11-05', 'No valid readings'),
 ('PM2.5 Adyar and Guindy today', 'Please specify exactly one'),
 ('PM2.5 Unknown today', 'Please specify exactly one'),
 ('PM2.5 Adyar on 2026-02-30', 'Invalid date'),
 ('Hello', 'Ask for PM2.5'),
 ('PM2.5 Adyar', 'Please specify today'),
]
answers=[]
for question,prefix in cases:
    answer=chatbot(question,DB,today=date(2026,10,1))
    assert answer.startswith(prefix)
    answers.append({'question':question,'answer':answer})
with __import__('sqlite3').connect(DB) as con:
    vals=[r[0] for r in con.execute('SELECT pm25 FROM readings WHERE station=? AND
        day=?',('Adyar','2026-10-01'))]
assert f'{sum(vals)/len(vals):.2f}' in answers[0]['answer']
text='Smoke covers the lane. The lane has smoke every night. Please inspect it.'
assert summarize(text) in sentences(text)
(ROOT/'chatbot_results.json').write_text(json.dumps({'clock_for_today':'2026-10-01 (fixed test
    date, not current readings)','cases':answers},indent=2))
print(json.dumps(answers,indent=2))
print('Eight chatbot cases and extractive-summary check: PASS')
```

TaskB4/compare_personal_review.py

```
"""Calculate actual student agreement only after all review fields are filled."""
import csv
import json
from pathlib import Path
from ai import label,sentences,summary_indices
ROOT=Path(__file__).resolve().parent

def main():
    with (ROOT/'personal_review.csv').open(newline='',encoding='utf-8') as f:
        rows=list(csv.DictReader(f))
    original=json.loads((ROOT/'complaints.json').read_text())
    expected={str(r['id']):r['text'] for r in original}
    seen=set();review=[]
    for row in rows:
        ident=row['id']
        if ident not in expected or ident in seen or row['complaint']!=expected[ident]:
            raise ValueError('Review IDs/text must match the original 50 complaints exactly.')
        seen.add(ident)
        sentiment=row['my_sentiment'].strip().lower()
        if sentiment not in {'positive','negative','neutral'}:
            raise ValueError(f'Complaint {ident}: enter your own positive/negative/neutral label.')
        try:idx=int(row['my_salient_sentence_number'])-1
        except ValueError as exc:raise ValueError(f'Complaint {ident}: enter your chosen one-based
            sentence number.') from exc
        if not 0<=idx<len(sentences(row['complaint'])):
            raise ValueError(f'Complaint {ident}: invalid sentence number.')
        review.append((label(row['complaint'])==sentiment,idx in
            summary_indices(row['complaint']),idx==0))
    if seen!=expected.keys():raise ValueError('Complete all 50 complaints.')
    print(json.dumps({'reviewed':len(review),'sentiment_agreement':sum(r[0] for r in review),
                      'frequency_salience_matches':sum(r[1] for r in review),
                      'first_sentence_salience_matches':sum(r[2] for r in review)},indent=2))
if __name__=='__main__':main()
```

TaskB4/complete_code.py

```
POSITIVE = {"good", "clean", "fast", "helpful", "improved"}
NEGATIVE = {"bad", "dirty", "slow", "rude", "smoke", "broken"}
NEGATORS = {"not", "never", "no"}
def score(text):
    """+1 per positive word, -1 per negative word; a preceding negator flips it."""
    words = text.lower().replace(".", " ").replace(",", " ").split()
    total = 0
    for i,word in enumerate(words):
        value = 1 if word in POSITIVE else -1 if word in NEGATIVE else 0
        if i > 0 and words[i-1] in NEGATORS:
            value = -value
        total += value
    return total

def label(text):
    s = score(text)
    return "positive" if s > 0 else "negative" if s < 0 else "neutral"

for c in ["Roads are clean and the bus was fast.",
          "Smoke from the factory, air is bad.",
          "The staff were not helpful.",
          "Not bad, service improved."]:
    print(f"{label(c):8} | {c}")
```

TaskB4/evaluate.py

```
import json
import statistics
import time
from pathlib import Path
from ai import label,score,sentences,summary_indices,summarize,first_k
ROOT=Path(__file__).resolve().parent

def main():
    rows=json.loads((ROOT/'complaints.json').read_text())
    assert len(rows)>=50
    assert label('not good')=='negative' and label('not bad')=='positive'
    assert summarize('',1)=='' and summarize('One sentence.',0)==''
    results=[]
    for r in rows:
        indices=summary_indices(r['text'])
        results.append({**r,'score':score(r['text']),'predicted_label':label(r['text']),
            'label_agrees':label(r['text'])==r['reference_label'],
            'frequency_summary':summarize(r['text']), 'first_sentence_summary':first_k(r['text']),
            'frequency_matches_reference':r['reference_salient_sentence'] in indices,
            'first_matches_reference':r['reference_salient_sentence']==0})
    timing={}
    for name,fn in
        [('sentiment',label),('frequency_summary',summarize),('first_sentence',first_k)]:
        runs=[]
        for _ in range(5):
            start=time.perf_counter()
            for r in rows:fn(r['text'])
            runs.append(time.perf_counter()-start)
        timing[name]={'runs_seconds':runs,'median_seconds_for_50':statistics.median(runs)}
    evidence={'complaints':len(rows),'label_agreement':sum(r['label_agrees'] for r in results),
              'frequency_salience_matches':sum(r['frequency_matches_reference'] for r in results),
              'first_sentence_salience_matches':sum(r['first_matches_reference'] for r in results),
              'reference_note':'Illustrative reference labels; not independently validated ground
                  truth.',
              'timings':timing,'results':results}
    (ROOT/'evaluation.json').write_text(json.dumps(evidence,indent=2))
    print(json.dumps({k:v for k,v in evidence.items() if k!='results'},indent=2))
    print('Known failures:')
    for r in results:
        if not r['label_agrees']:print(f"{r['id']:02}: reference={r['reference_label']},
            predicted={r['predicted_label']} | {r['text']}")
if __name__=='__main__':main()
```

TaskB4/index_experiment.py

```
"""Controlled before/after index trial on B3's initially unindexed table."""
import argparse
import json
import shutil
import sqlite3
import statistics
import time
from pathlib import Path
ROOT=Path(__file__).resolve().parent
QUERIES={
 'station_month':('SELECT AVG(pm25) FROM aq WHERE station=? AND day>=? AND
     day<?',('S001','2026-10-01','2026-11-01')),
 'station_comparison':('SELECT station,AVG(pm25) FROM aq WHERE day>=? AND day<? GROUP BY station
     ORDER BY AVG(pm25) DESC',('2026-10-01','2026-11-01')),
 'worst_hours':('SELECT hour,AVG(pm25) FROM aq WHERE day>=? AND day<? GROUP BY hour ORDER BY
     AVG(pm25) DESC LIMIT 5',('2026-10-01','2026-11-01')),
}
def measure(con):
    result={}
    for name,(sql,args) in QUERIES.items():
        con.execute(sql,args).fetchall() # warm-up, not timed
        runs=[]
        for _ in range(7):
            start=time.perf_counter();rows=con.execute(sql,args).fetchall();runs.append(time.perf_c
                ounter()-start)
        result[name]={'runs_seconds':runs,'median_seconds':statistics.median(runs),'result':rows,
                      'plan':con.execute('EXPLAIN QUERY PLAN '+sql,args).fetchall()}
    return result

def main(source):
    dest=source.with_name('index_experiment.sqlite');shutil.copy2(source,dest)
    with sqlite3.connect(dest) as con:
        before=measure(con)
        slowest=max(before,key=lambda k:before[k]['median_seconds'])
        start=time.perf_counter();con.execute('CREATE INDEX idx_station_day ON
            aq(station,day)');con.commit();build=time.perf_counter()-start
        after=measure(con)
        for name in before:
            # Equal query results, allowing harmless changes in summation order.
            a,b=before[name]['result'],after[name]['result']
            assert len(a)==len(b)
            for x,y in zip(a,b):
                for u,v in zip(x,y):
                    if isinstance(u,float):assert abs(u-v)<1e-8
                    else:assert u==v
    result={'rows':1051200,'repetitions':7,'baseline_slowest':slowest,'index_build_seconds':build,
            'before':before,'after':after,'note':'B3 heap table. B1 already has a composite
                primary-key index with this prefix; do not add a redundant production index.'}
    (ROOT/'index_results.json').write_text(json.dumps(result,indent=2))
    print('Baseline slowest query:',slowest)
    for name in before:
        print(f"{name}: {before[name]['median_seconds']:.6f}s ->
            {after[name]['median_seconds']:.6f}s; ratio
            {before[name]['median_seconds']/after[name]['median_seconds']:.2f}x")
    print(f'Index construction: {build:.6f}s')
if __name__=='__main__':
    p=argparse.ArgumentParser();p.add_argument('database',type=Path);main(p.parse_args().database)
```

verify.py

```
"""Focused regression and saved-evidence checks; run after the task scripts."""
import csv
import json
import math
import sqlite3
import subprocess
import sys
import tempfile
import unittest
from pathlib import Path
ROOT=Path(__file__).resolve().parent
sys.path.insert(0,str(ROOT/'TaskB1'))
sys.path.insert(0,str(ROOT/'TaskB4'))
from pipeline import clean_file, min_max, z_scores, FIELDS, SourceError, store
from ai import label,summarize,chatbot

class AssignmentChecks(unittest.TestCase):
    def test_original_exercises(self):
        expected={
            'TaskB1':"[('Adyar', '2026-10-01', 48.0), ('Guindy', '2026-10-01', 95.0), ('Guindy',
                '2026-10-03', 121.0)]\n[0.0, 0.644, 1.0]\nAbove-999 row skipped: PASS\n",
            'TaskB2':'54.0 108.0\nNone\n',
            'TaskB4':'positive | Roads are clean and the bus was fast.\nnegative | Smoke from the
                factory, air is bad.\nnegative | The staff were not helpful.\npositive | Not bad,
                service improved.\n'}
        for task,output in expected.items():
            with self.subTest(task=task):
                result=subprocess.run([sys.executable,str(ROOT/task/'complete_code.py')],text=True,
                    capture_output=True,check=True)
                self.assertEqual(result.stdout,output)

    def test_cleaned_fixture_and_scaling(self):
        rows,issues=clean_file(ROOT/'TaskB1/small_messy.csv')
        self.assertEqual(len(rows),4);self.assertEqual(len(issues),17)
        self.assertEqual(rows[2],('Guindy','2026-10-01',0,95.,150.,25.,30.))
        self.assertEqual(min_max([4,6,8]),[0,0.5,1])
        zs=z_scores([4,6,8])
        self.assertAlmostEqual(sum(zs),0)
        self.assertAlmostEqual(sum(z*z for z in zs)/3,1)
        for fn in [min_max,z_scores]:
            self.assertEqual(fn([]),[]);self.assertEqual(fn([7,7]),[0,0])

    def test_invalid_row_does_not_reserve_key(self):
        with tempfile.TemporaryDirectory() as tmp:
            p=Path(tmp)/'data.csv'
            with p.open('w',encoding='utf-8-sig',newline='') as f:
                w=csv.writer(f);w.writerow(FIELDS)
                w.writerows([['Adyar','2026-10-01',0,-1,80,10,30,'C'],
                             ['Adyar','2026-10-01',0,50,80,10,30,'C'],
                             ['Adyar','2026-10-01',0,99,80,10,30,'C']])
            rows,issues=clean_file(p)
            self.assertEqual(len(rows),1);self.assertEqual(rows[0][3],50)
            self.assertEqual(len(issues),2)

    def test_corrupt_files_fail_explicitly(self):
        for p in (ROOT/'TaskB2/inputs').glob('*.csv'):
            with self.subTest(file=p.name),self.assertRaises(SourceError):clean_file(p)

    def test_database_constraints_and_independent_averages(self):
        rows,_=clean_file(ROOT/'TaskB1/small_messy.csv')
        with tempfile.TemporaryDirectory() as tmp:
            db=Path(tmp)/'small.sqlite';store(rows,db)
            with sqlite3.connect(db) as con:
                self.assertEqual(con.execute("SELECT AVG(pm25),COUNT(*) FROM readings WHERE
                    station='Adyar'").fetchone(),(54.0,2))
                self.assertEqual(con.execute("SELECT minimum,maximum,mean,valid_count FROM
                    daily_summary WHERE station='Adyar'").fetchone(),(48.0,60.0,54.0,2))
                with self.assertRaises(sqlite3.IntegrityError):con.execute('INSERT INTO readings
                    VALUES (?,?,?,?,?,?,?)',rows[0])
                con.execute('PRAGMA foreign_keys=ON')
        with sqlite3.connect(ROOT/'TaskB1/air_quality.sqlite') as con:
            self.assertEqual(con.execute('PRAGMA integrity_check').fetchone()[0],'ok')
            self.assertEqual(con.execute('SELECT COUNT(*) FROM readings').fetchone()[0],8880)
            self.assertEqual(con.execute('SELECT COUNT(*) FROM daily_summary').fetchone()[0],372)

    def test_saved_benchmark_evidence(self):
        for filename,counts in [('TaskB1/cleaning_benchmarks.json',[1000,100000,1000000]),('TaskB3/
            measurements.json',[8760,105120,1051200])]:
```

```
            data=json.loads((ROOT/filename).read_text())
            self.assertEqual([r['rows'] for r in data],counts)
            for size in data:
                checks=[]
                for method in size['methods'].values():
                    self.assertEqual(len(method['runs']),3)
                    for run in method['runs']:
                        self.assertGreater(run['seconds'],0)
                        checks.append(run['check'])
                for check in checks:
                    for key in
                    checks[0]:self.assertTrue(math.isclose(check[key],checks[0][key],rel_tol=1e-
                    9,abs_tol=1e-7))

    def test_ai_expected_scope_and_personal_review(self):
        self.assertEqual(label('not good'),'negative');self.assertEqual(label('not
            bad'),'positive')
        self.assertEqual(summarize(''), '')
        text='Smoke fills the lane. Smoke enters the lane every evening. Please inspect it.'
        self.assertIn(summarize(text),text)
        result=subprocess.run([sys.executable,str(ROOT/'TaskB4/compare_personal_review.py')],text=T
            rue,capture_output=True)
        self.assertNotEqual(result.returncode,0)
        self.assertIn('enter your own',result.stderr)
        evidence=json.loads((ROOT/'TaskB4/evaluation.json').read_text())
        self.assertEqual(evidence['complaints'],50)
        self.assertEqual(evidence['label_agreement'],31)

    def test_chatbot_no_data_and_invalid_dates(self):
        db=ROOT/'TaskB1/air_quality.sqlite'
        self.assertTrue(chatbot('PM2.5 Adyar 2026-11-05',db).startswith('No valid readings'))
        self.assertTrue(chatbot('PM2.5 Adyar 2026-02-30',db).startswith('Invalid date'))
        self.assertTrue(chatbot("PM2.5 King's Park 2026-10-01",db).startswith("King's Park on
            2026-10-01:"))

if __name__=='__main__':unittest.main(verbosity=2)
```